Smart Checkout: Retail Product Detection and Automatic Checkout

Shuang Dai
Bachelor's Degree Programme in Informatica
Course: AI-LAB – Computer Vision, Signal Processing, and Natural Language Processing
Department of Computer Science, Sapienza University of Rome
Matricola: 2104150
dai.2104150@studenti.uniroma1.it

Abstract

Automatic checkout is a challenging computer vision application because a system must recognize multiple fine-grained retail products in a single image, localize them accurately, and convert the detections into a reliable shopping list. This project develops a complete automatic checkout pipeline based on the Retail Product Checkout (RPC) dataset and Ultralytics YOLO11. The multi-product checkout scenes from the original validation and test sets are combined and reorganized into leakage-aware grouped training, validation, and test splits. COCO-format annotations are converted into the normalized YOLO format, and two detectors are evaluated: YOLO11n as a lightweight baseline and YOLO11s as the main model. Beyond standard object-detection metrics, the project introduces a strict checkout-accuracy metric that considers an image correct only when the predicted quantity of every SKU exactly matches the ground truth. The final system converts detections into product quantities, looks up simulated unit prices, calculates line totals and the final amount, and generates an English checkout receipt. On the custom test split, YOLO11s achieves 0.9954 precision, 0.9956 recall, 0.9948 mAP@0.5, 0.8869 mAP@0.5:0.95, and 94.91% strict checkout accuracy. YOLO11n achieves 92.94% strict checkout accuracy with lower inference latency. The results show that a lightweight YOLO-based detector can support a complete automatic checkout workflow while also revealing the remaining difficulty of exact SKU-level counting in multi-product scenes.

Keywords: computer vision; object detection; retail product recognition; automatic checkout; YOLO11; deep learning; product counting.

1 Introduction

Automatic checkout (ACO) aims to reduce or remove the need for manual barcode scanning by using computer vision to recognize the products placed at a checkout station. This problem is particularly challenging in retail environments because products can be visually similar, appear at different scales and orientations, overlap with one another, and occur in large numbers within the same image. The RPC dataset was introduced specifically to support research on automatic checkout and fine-grained retail product recognition, combining isolated product images with realistic multi-product checkout scenes [1].

Traditional retail recognition systems often depend on controlled acquisition conditions, barcode readers, or handcrafted visual matching. Such approaches become less convenient when several products are simultaneously visible or when products are partially occluded. Modern object detectors instead learn both object localization and category prediction directly from images. YOLO formulated detection as a single end-to-end prediction problem and demonstrated the advantages of single-stage detection for fast inference [3]. Other detectors, such as Faster R-CNN [4] and SSD [5], established important alternatives based on region proposals and multi-scale single-stage prediction, respectively.

The goal of this project is to build an end-to-end Smart Checkout system rather than only training an object detector. The system detects and identifies products, counts repeated SKUs, evaluates whether the complete shopping list is correct, retrieves a predefined unit price for each product, computes the total cost, and generates a checkout receipt. The main personal contribution is the construction of a reproducible experimental pipeline around the RPC checkout scenes: COCO-to-YOLO conversion, leakage-aware grouped splitting, comparison of YOLO11n and YOLO11s, validation-based confidence-threshold selection, strict SKU-level checkout accuracy, and integration of detection results with a price lookup and receipt-generation stage.

The remainder of the report is organized as follows. Section 2 discusses related work and the RPC benchmark. Section 3 describes the proposed methodology and complete checkout pipeline. Section 4 presents the dataset, training protocol, quantitative results, qualitative checkout examples, and critical discussion. Section 5 concludes the report and discusses future improvements.

2 Related Work

Early object-recognition systems frequently relied on handcrafted visual features and matching strategies. Local feature descriptors such as SIFT were designed to provide robustness to scale and rotation changes and became an important foundation for image matching [6]. In a retail environment, however, handcrafted matching can become difficult when multiple products are present, packaging changes, or objects overlap. Modern learned detectors address these limitations by learning representations jointly with localization and classification.

Faster R-CNN introduced a Region Proposal Network that

shares convolutional features with the detection network, providing a strong two-stage object-detection framework [4]. SSD subsequently demonstrated that object detection can be performed efficiently in a single network without a separate proposal-generation stage [5]. YOLO further emphasized end-to-end single-stage detection, predicting bounding boxes and class probabilities directly from the full image [3]. These developments are particularly relevant to automatic checkout because a checkout station requires simultaneous localization of many objects and benefits from efficient inference.

The retail domain introduces an additional challenge: fine-grained SKU recognition. Products that belong to the same broad category may have highly similar packaging while corresponding to different SKUs. Wei et al. introduced the Retail Product Checkout (RPC) dataset to address this problem. The dataset contains controlled single-product exemplar images and multi-product checkout images with object-level annotations, making it suitable for studying both recognition and realistic checkout scenarios [1]. The official dataset contains 53,739 exemplar training images, 6,000 checkout validation images, and 24,000 checkout test images, with 200 product categories in the fine-grained SKU setting [2].

This project uses the checkout component of RPC rather than treating isolated product recognition as the final task. The main difference from a standard object-detection benchmark is the application-level evaluation: an automatic checkout is considered correct only when the complete predicted SKU inventory matches the ground truth for the image. This strict metric is closer to the actual purpose of the system than reporting mAP alone. The project also compares the lightweight YOLO11n model with the larger YOLO11s model. YOLO11 is a recent Ultralytics detector designed for an accuracy-efficiency trade-off across computer vision tasks [7].

3 Methodology

The proposed system is organized as a complete pipeline from a multi-product checkout image to a final electronic receipt. Figure 1 summarizes the main stages.

3.1 Dataset Preparation and Annotation Conversion

The project uses the RPC multi-product checkout scenes. Instead of using the isolated training images as the primary experimental data, the original validation and test checkout scenes are combined and reorganized into a new custom split. This design provides a large number of realistic multi-product images while allowing the complete experimental protocol to be controlled.

The original annotations are provided in COCO-style JSON format. For YOLO training, every bounding box is converted from the COCO representation (x, y, w, h) into normalized YOLO coordinates:

$$x_c = \frac{x + w/2}{W}, \qquad y_c = \frac{y + h/2}{H},$$

$$w_n = \frac{w}{W}, \qquad h_n = \frac{h}{H},$$

where W and H are the image width and height. Each label is stored as `class_id x_center y_center width height`.

The conversion procedure creates separate image and label directories for the training, validation, and test subsets. Hard links are used to reuse the original images without creating unnecessary duplicate copies of the large dataset. The converted dataset is also visualized before training to verify that the bounding boxes remain aligned with the products.

3.2 Leakage-Aware Grouped Splitting

A major methodological concern in checkout datasets is data leakage between visually similar captures. Images recorded close to each other may share the same checkout arrangement or camera conditions. A random image-level split could therefore make the test set artificially similar to the training data.

To reduce this risk, a capture-group identifier is derived from the RPC filenames. Images assigned to the same group are kept in the same subset. GroupShuffleSplit is then used to create approximately 80% training, 10% validation, and 10% test partitions. The resulting split contains 23,985 training images, 2,970 validation images, and 3,045 test images. The corresponding numbers of annotated product instances are 294,410, 36,568, and 36,957.

3.3 YOLO11 Models

Two Ultralytics YOLO11 object detectors are evaluated. YOLO11n is used as the lightweight baseline, while YOLO11s is used as the main model. Both models are initialized from pretrained weights and fine-tuned on the converted RPC checkout dataset. The use of pretrained weights reduces the amount of training required to learn generic visual features while allowing the detector to specialize in the 200 retail SKU classes.

According to the observed model summaries, YOLO11n contains approximately 2.68 million parameters and requires about 6.9 GFLOPs for the configured detection model, whereas YOLO11s contains approximately 9.49 million parameters and requires about 21.8 GFLOPs. This difference makes the two models useful for studying the trade-off between accuracy and inference speed.

3.4 Checkout-Level Post-Processing

Object detection produces individual SKU predictions. The system converts these predictions into a shopping list by grouping detections according to their category identifiers. If the model predicts the same SKU multiple times, each detection contributes one unit to the corresponding quantity.

A predefined product metadata table provides an English product name and a simulated unit price for each SKU. The price table contains all 200 SKUs and uses CNY as the reporting currency. The price values are explicitly treated as simulated values rather than prices learned from images or retrieved from a live retailer database.

For each SKU i, the line total is computed as

$$L_i = q_i p_i,$$

where q_i is the detected quantity and p_i is the predefined unit price. The final checkout amount is

$$T = \sum_i q_i p_i.$$

The system then generates an English receipt containing product names, unit prices, quantities, line totals, and the final amount.

3.5 Checkout Accuracy

Standard object-detection metrics do not directly answer whether the system can successfully complete a checkout. Therefore, an application-level strict checkout accuracy is introduced.

For each image, let C_i be the ground-truth count of SKU i and $\hat{C}_i$ the predicted count. The checkout is considered correct only when

$$\hat{C}_i = C_i \quad \text{for every SKU } i.$$

The strict checkout accuracy is then

$$cAcc = \frac{N_{\text{exact}}}{N},$$

where N_{exact} is the number of images with an exactly matching SKU-count vector and N is the number of evaluated images.

The confidence threshold is selected using the validation split. Candidate thresholds from 0.05 to 0.50 are evaluated, and the threshold maximizing strict checkout accuracy is preferred, with mean count error used as a secondary criterion. The selected threshold is then applied once to the untouched test set.

4 Experimental Setup

4.1 Dataset

The original RPC dataset contains 83,739 images and 200 fine-grained product categories. Its official structure contains 53,739 single-product exemplar images, 6,000 multi-product validation images, and 24,000 multi-product test images [2]. The present experiment focuses on the multi-product checkout scenes from the validation and test sets.

After combining these checkout scenes, the experiment contains 30,000 images and 367,935 annotated product instances. There are 200 SKU classes, with an average of approximately 12.25 products per image.

The original RPC dataset has different clutter levels in its checkout images. This is important because increasing the number of overlapping or visually similar products makes exact SKU recognition and counting more difficult. The custom split preserves the multi-product nature of the data but does not reproduce the official RPC benchmark protocol; therefore, the reported values should be interpreted as results for the custom experimental split rather than official RPC leaderboard scores.

Table 1: Dataset summary used in the custom experiment.

Property	**Value**
Dataset	RPC
Checkout images	30,000
Product instances	367,935
SKU classes	200
Training images	23,985
Validation images	2,970
Test images	3,045
Train instances	294,410
Validation instances	36,568
Test instances	36,957
Mean products/image	12.25
Annotation format	COCO JSON
Training format	YOLO TXT

4.2 Training and Evaluation Protocol

The implementation uses Python, PyTorch, Ultralytics YOLO, OpenCV, NumPy, pandas, Matplotlib, Seaborn, PyYAML, tqdm, and scikit-learn. The recorded runtime uses PyTorch 2.8.0 with CUDA 12.8 and an NVIDIA GeForce RTX 4090 GPU. Ultralytics version 8.4.126 is used for the experiments.

Both models use the same dataset and training configuration. The main training parameters are summarized in Table 2.

Table 2: Training configuration.

Parameter	**Value**
Framework	PyTorch + Ultralytics YOLO
Baseline	YOLO11n
Main model	YOLO11s
Optimizer	AdamW
Initial learning rate	0.001
Batch size	64
Image size	640×640
Epochs	10
Detection losses	Box + classification + DFL
Main metrics	mAP@0.5:0.95, cAcc
Random seed	42

A fixed random seed of 42 is used for reproducibility. The same grouped split and training settings are used for both models. The baseline/main comparison therefore isolates the effect of model scale as much as possible.

4.3 Results

Table 3 reports the main quantitative results on the custom test split. YOLO11s provides a small but consistent improvement over YOLO11n in all reported detection metrics and in strict checkout accuracy.

YOLO11n achieves 99.51% precision and 99.39% recall, with an mAP@0.5 of 99.46% and an mAP@0.5:0.95 of 87.74%. Its strict checkout accuracy is 92.94%. YOLO11s improves precision to 99.54%, recall to 99.56%, mAP@0.5 to 99.48%, and mAP@0.5:0.95 to 88.69%. More impor-

Table 3: Comparison between YOLO11n and YOLO11s on the custom test split.

Model	Precision	Recall	mAP@0.5	mAP@0.5:0.95	cAcc	Latency (ms)
YOLO11n	0.9951	0.9939	0.9946	0.8774	0.9294	0.7260
YOLO11s	**0.9954**	**0.9956**	**0.9948**	**0.8869**	**0.9491**	1.4308

tantly for the application, strict checkout accuracy increases to 94.91%.

The main model is therefore more reliable at producing a complete SKU-count vector, although its average measured inference time is approximately 1.43 ms compared with 0.73 ms for YOLO11n in the recorded GPU evaluation. This represents roughly a twofold latency increase. The comparison illustrates that the larger model provides a measurable application-level benefit, but the lightweight model remains attractive when inference efficiency is prioritized.

The difference between the very high mAP@0.5 values and the lower strict checkout accuracy is particularly informative. A detector can achieve high localization and classification scores while still making a small number of mistakes in individual images. In an automatic checkout system, a single missed or duplicated product is enough to make the complete receipt incorrect. Therefore, strict checkout accuracy is a more demanding metric for the final application.

4.4 Critical Discussion

The experiments support three main conclusions. First, YOLO11 is highly effective for the fine-grained multi-product detection task on the custom RPC checkout split. Both tested models achieve precision and recall above 99%, indicating that the detector generally localizes and identifies the visible products correctly.

Second, model size matters more for the application-level checkout metric than for conventional mAP@0.5. YOLO11s improves strict checkout accuracy from 92.94% to 94.91%, a gain of 2.0 percentage points, while the improvement in mAP@0.5 is very small. This shows why an application-specific metric is useful: the final goal is not only to detect objects, but to recover the exact shopping list.

Third, the lightweight YOLO11n model remains a strong baseline. Its inference time is approximately half that of YOLO11s in the recorded GPU benchmark, while still reaching 92.94% exact checkout accuracy. For a real-time or resource-constrained deployment, this trade-off may be preferable.

There are several limitations. The experiment uses only 10 training epochs, so the models may not be fully optimized. The custom split is constructed from the original RPC validation and test scenes and therefore must not be presented as an official RPC benchmark. The grouping strategy reduces leakage from adjacent captures, but the derived grouping rule is project-specific. In addition, the price database is simulated and cannot represent current real-world supermarket prices. The receipt-generation stage therefore demonstrates the integration of computer vision with a point-of-sale style database rather than a real commercial checkout service.

The system can also fail when products are heavily occluded, visually similar SKUs are adjacent, or a small detection error changes the quantity of one SKU. Future experiments could increase the number of training epochs, evaluate additional confidence and IoU settings, analyze performance separately by clutter level, and compare against another detector family. A further improvement would be to connect the recognized SKU identifiers to a real product database containing validated names, barcodes, prices, and availability information.

5 Conclusion

This project developed a complete Smart Checkout pipeline for fine-grained retail product recognition. Starting from multi-product RPC checkout images, the project converts COCO annotations into YOLO format, constructs leakage-aware grouped train/validation/test splits, fine-tunes YOLO11 detectors, and evaluates both object-level detection quality and application-level checkout correctness.

The YOLO11s main model achieves 0.9954 precision, 0.9956 recall, 0.9948 mAP@0.5, 0.8869 mAP@0.5:0.95, and 94.91% strict checkout accuracy on the custom test split. Compared with the YOLO11n baseline, the larger model provides better exact checkout performance at the cost of approximately doubled inference latency.

The main contribution of the project is not a new detection architecture, but an end-to-end and reproducible application pipeline that connects object detection with SKU counting, checkout verification, price lookup, and receipt generation. This demonstrates how a standard deep-learning detector can be transformed into a practical computer-vision application.

The current system should be regarded as a prototype rather than a production checkout solution. In particular, the price data are simulated, the custom evaluation protocol differs from the official RPC benchmark, and the relatively short training schedule limits the strength of the conclusions. Nevertheless, the results demonstrate that fine-grained object detection can provide a strong foundation for automatic checkout and that exact shopping-list accuracy is an important complement to conventional detection metrics.

References

[1] X.-S. Wei, Q. Cui, L. Yang, P. Wang, L. Liu, and J. Yang, "RPC: A Large-Scale and Fine-Grained Retail Product Checkout Dataset," arXiv preprint arXiv:1901.07249, 2019. https://arxiv.org/abs/1901.07249

[2] RPC Dataset Project, "RPC: A Large-Scale Retail Product Checkout Dataset," 2019. https://rpc-dataset.github.io/

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Ob-

ject Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016. https://doi.org/10.1109/CVPR.2016.91

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015. https://arxiv.org/abs/1506.01497

[5] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. Berg, "SSD: Single Shot MultiBox Detector," in *European Conference on Computer Vision (ECCV)*, pp. 21–37, 2016. https://doi.org/10.1007/978-3-319-46448-0_2

[6] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, pp. 91–110, 2004. https://doi.org/10.1023/B:VISI.0000029664.99615.94

[7] Ultralytics, "Ultralytics YOLO11 Documentation," 2024–2026. https://docs.ultralytics.com/models/yolo11

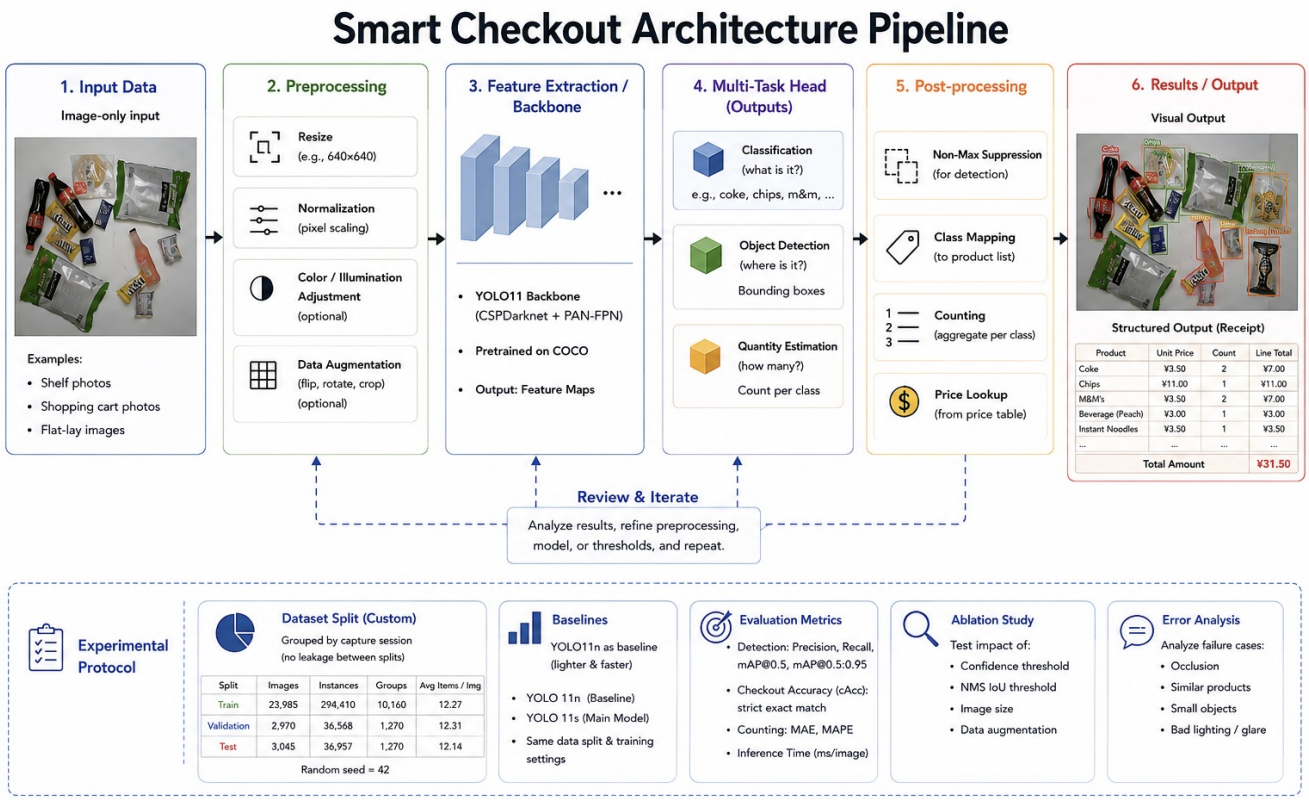


Figure 1: Complete Smart Checkout pipeline used in the project. Starting from a single multi-product image, the system performs image preprocessing, product detection and recognition, SKU counting, price lookup, line-total calculation, and final checkout amount generation.